CSE331
AUTOMATA AND COMPUTABILITY

Turing Machines: decidable and recognizable languages

Lecture 15

Spring 2026

Today's learning goals

Sipser Ch 3.1

- Design TMs using different levels of descriptions
- Give high-level description for TMs used in constructions
- Prove properties of the classes of recognizable and decidable sets.

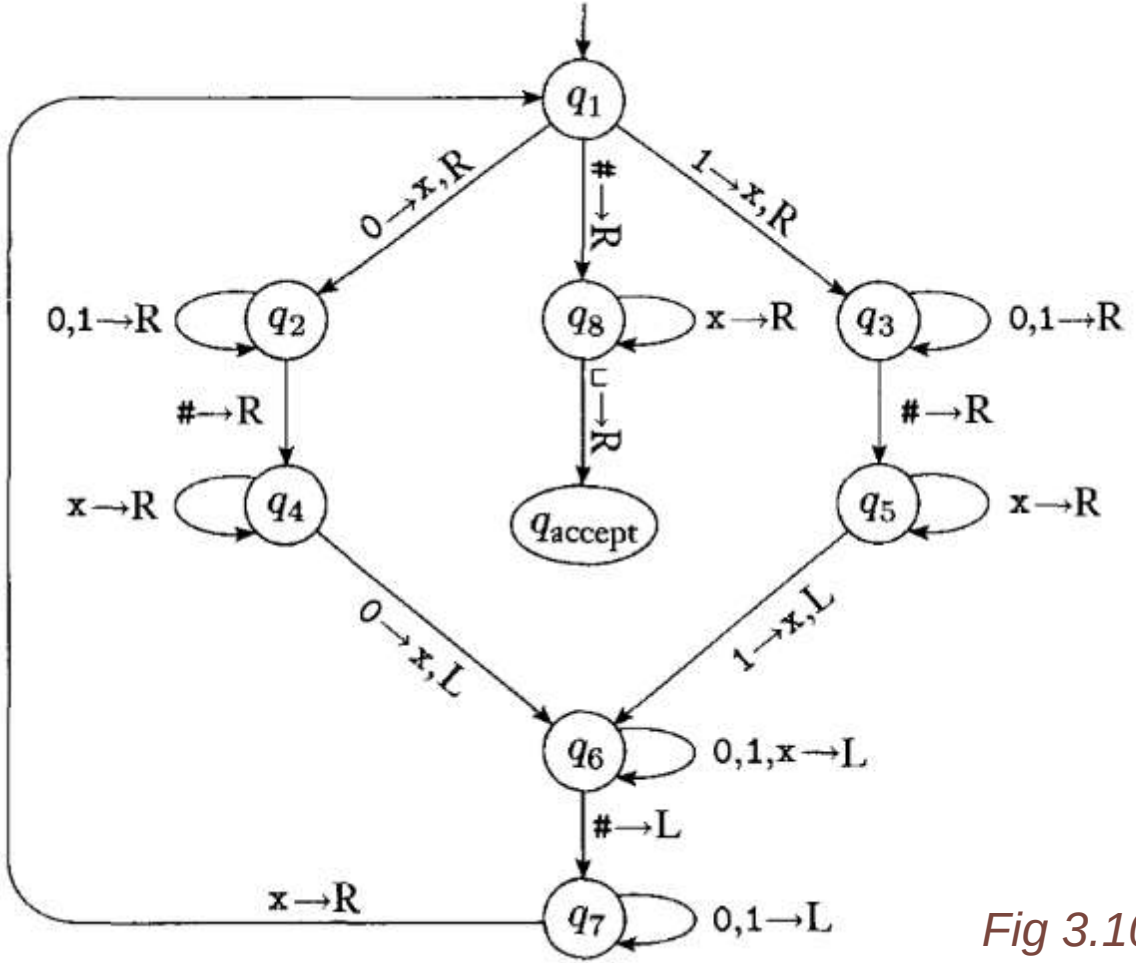
An example

$$L = \{ w\#w \mid w \text{ is in } \{0,1\}^* \}$$

Idea for Turing machine

- Zig-zag across tape to corresponding positions on either side of '#' to check whether these positions agree. If they do not, or if there is no '#', *reject*. If they do, cross them off.
- Once all symbols to the left of the '#' are crossed off, check for any symbols to the right of '#': if there are any, *reject*; if there aren't, *accept*.

How would you use this machine to prove that L is **decidable**?



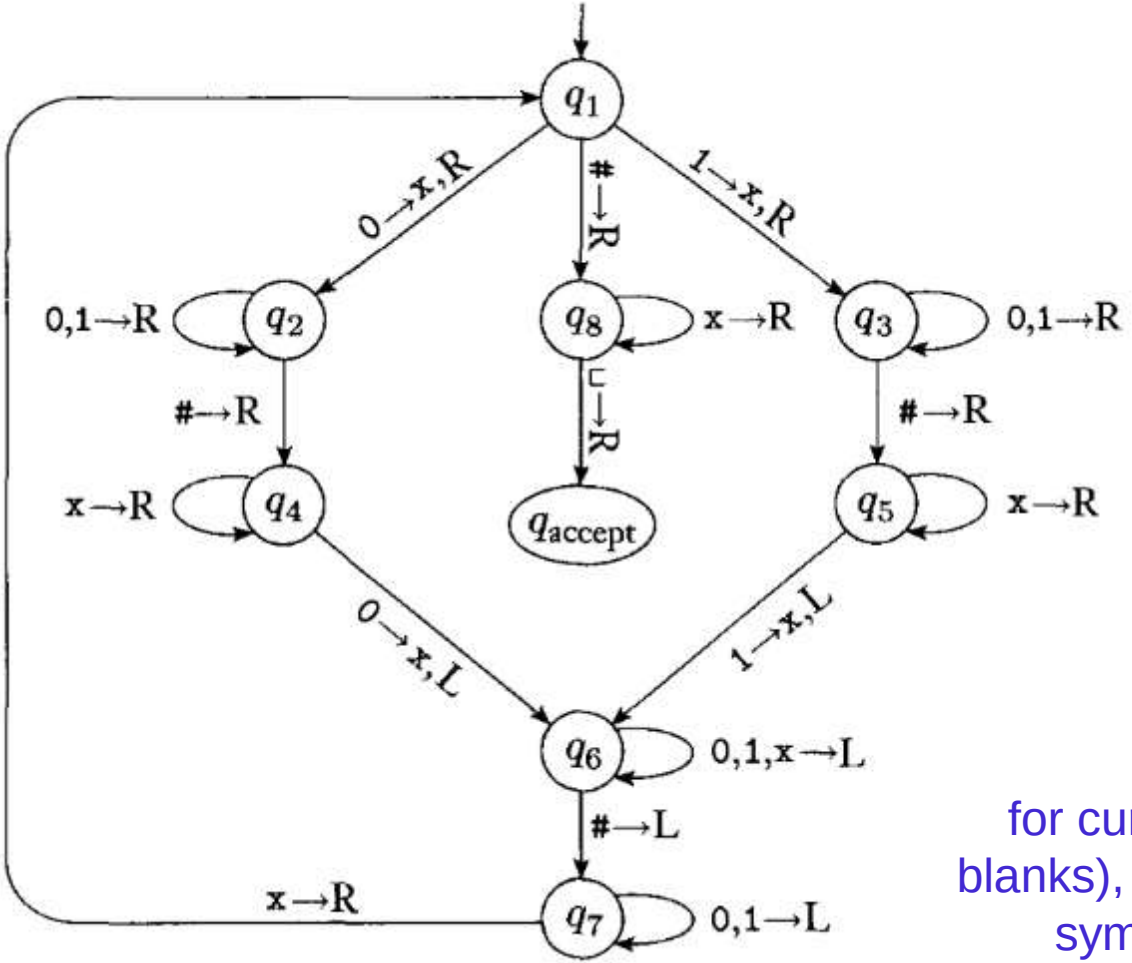
Q={q1,q2,q3,q4,q5,q6,q7, q8,q_{accept},q_{reject}}

Σ = {0,1,#}

Γ = {0,1,#,x,_ }

All missing transitions have output (q_{reject}, _, R)

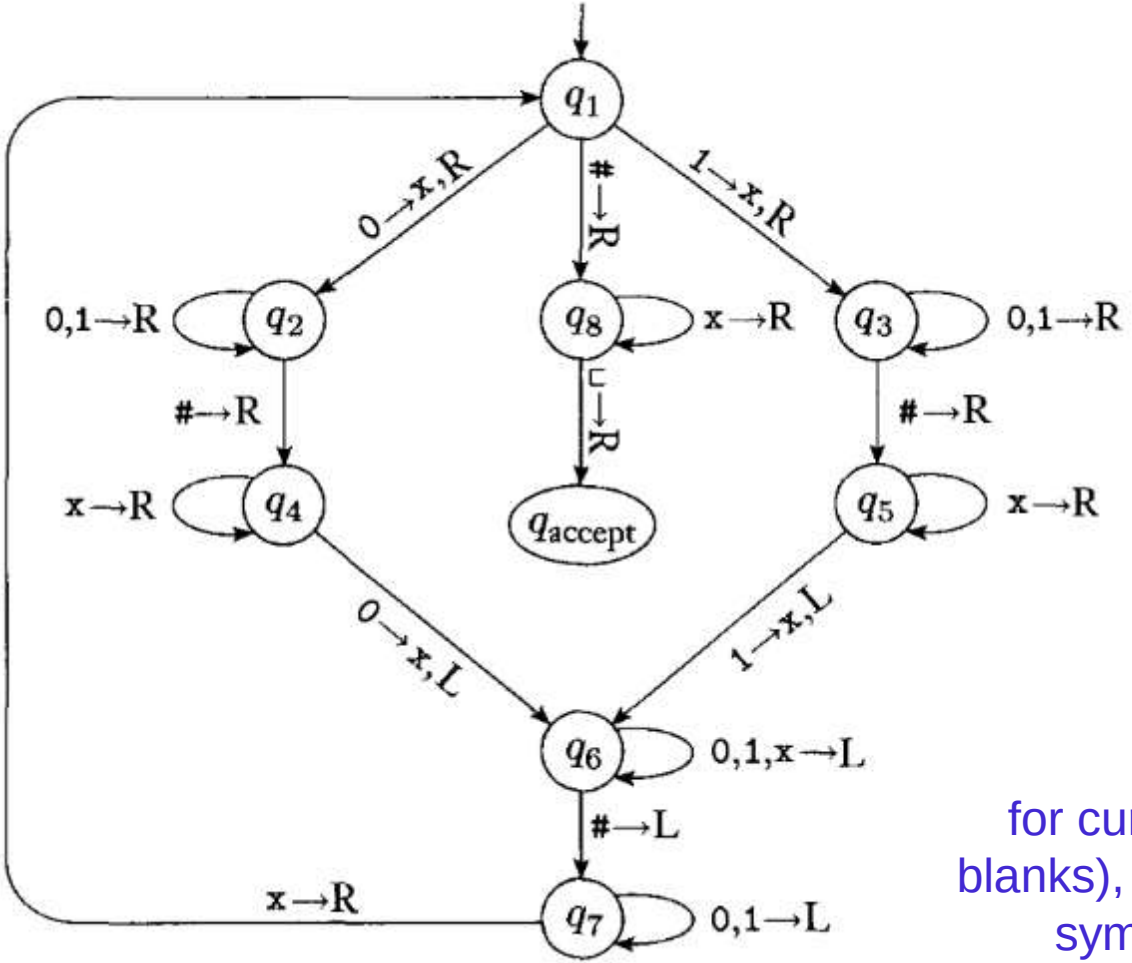
Fig 3.10 in Sipser



Computation on input 0#0?

Configuration **u q v**
for current tape uv (and then all blanks), current head location is first symbol of v, current state q

Computation on input 0# ?



Configuration **u q v**
for current tape uv (and then all blanks), current head location is first symbol of v, current state q

Describing TMs

Sipser p. 159

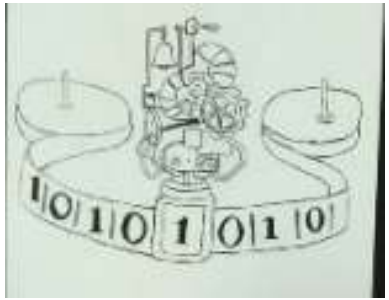
- **Formal definition**: set of states, input alphabet, tape alphabet, transition function, state state, accept state, reject state.
- **Implementation-level definition**: English prose to describe Turing machine head movements relative to contents of tape.
- **High-level desciption**: Description of algorithm, without implementation details of machine. As part of this description, can "call" and run another TM as a subroutine.

An example

Which of the following is an implementation-level description of a TM which **decides the empty set**?

M = "On input w:

A. reject."
B. sweep right across the tape until find a non-blank symbol. Then, reject."
C. If the first tape symbol is blank, accept. Otherwise, reject."
D. More than one of the above.
E. I don't know.

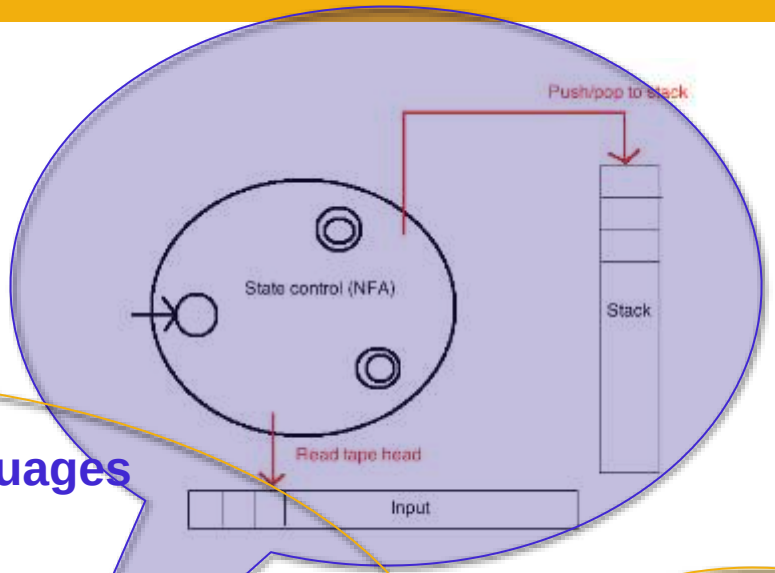


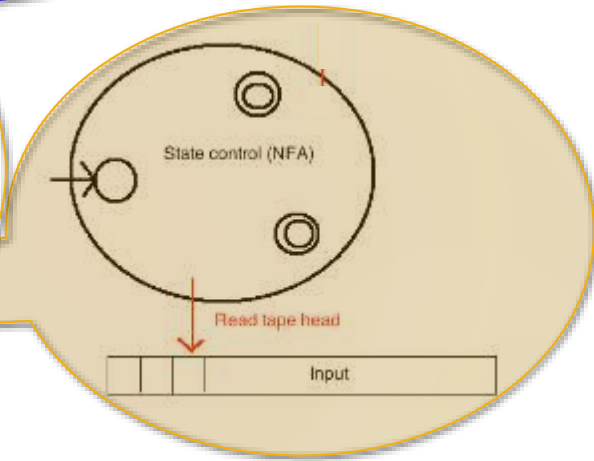
Turing recognizable languages

Turing decidable languages

Context-free languages

Regular languages





Decidable vs Recognizable

- Let M be a TM

- M decides a language L if:
  - If $w \in L$ then $M(w)$ accepts
  - If $w \notin L$ then $M(w)$ rejects

M always halts!

- M recognizes a language L if:
  - If $w \in L$ then $M(w)$ accepts
  - If $w \notin L$ then $M(w)$ rejects or loops (runs forever)

Decidable vs Recognizable

	M decides L	M recognizes L
$w \in L$	$M(w)$ accepts	$M(w)$ accepts
$w \notin L$	$M(w)$ rejects	$M(w)$ rejects or loops

Closure

Theorem: The class of decidable languages over fixed alphabet Σ is closed under union.

Proof: Let …

WTS …

Closure

Theorem: The class of decidable languages over fixed alphabet Σ is closed under union.

Proof: Let L_1 and L_2 be languages over Σ and suppose M_1 and M_2 are TMs deciding these languages. We will **define** a new TM, M, via a high-level description. We will then show that **$L(M) = L_1 \cup L_2$ and that M always halts.**

Closure

Theorem: The class of decidable languages over fixed alphabet Σ is closed under union.

Proof: Let L_1 and L_2 be languages and suppose M_1 and M_2 are TMs deciding these languages.

Construct the TM M as "On input w,

1. Run M_1 on input w. If M_1 accepts w, accept. Otherwise, go to 2.
2. Run M_2 on input w. If M_2 accepts w, accept. Otherwise, reject."

Correctness of construction:

WTS L(M) = L_1 U L_2 and M is a decider.

Where do we use decidability?

Closure proof

- WTS L(M) = L_1 U L_2 and M is a decider.

(1) If w in L_1:
- M1(w) accepts; so, M accepts w

(2) If w in L_2:
- If M1(w) accepts, M accepts w
- If not, it rejects, and then M2(w) accepts.
- Either way, M accepts w.

(3) If w not in L_1 U L_2:
- M1(w) rejects
- M2(w) rejects
- So, M rejects w.

M(w):
1. Run M1(w). If accepts, accept.
2. Run M2(w). If accepts, accept, otherwise reject.

Closure proof

- WTS L(M) = L_1 U L_2 and M is a decider.

(1) If w in L_1:
- M1(w) accepts; so, M accepts w

(2) If w in L_2:
- If M1(w) accepts, M accepts w
- If not, it rejects, and then M2(w) accepts.
- Either way, M accepts w.

(3) If w not in L_1 U L_2:
- M1(w) rejects
- M2(w) rejects
- So, M rejects w.

M(w):
1. Run M1(w). If accepts, accept.
2. Run M2(w). If accepts, accept, otherwise reject.

M is a decider because
it accepts or rejects all inputs

Closure proof

- WTS L(M) = L_1 U L_2 and M is a decider.

(1) If w in L_1:
- M1(w) accepts; so, M accepts w

(2) If w in L_2:
- If M1(w) accepts, M accepts w
- If not, it rejects, and then M2(w) accepts.
- Either way, M accepts w.

(3) If w not in L_1 U L_2:
- M1(w) rejects
- M2(w) rejects
- So, M rejects w.

M(w):
1. Run M1(w). If accepts, accept.
2. Run M2(w). If accepts, accept, otherwise reject.

Where do we use decidability of M1, M2?

Closure proof

- WTS L(M) = L_1 U L_2 and M is a decider.

(1) If w in L_1:
- M1(w) accepts; so, M accepts w

(2) If w in L_2:
- If M1(w) accepts, M accepts w
- If not, it rejects, and then M2(w) accepts.
- Either way, M accepts w.

(3) If w not in L_1 U L_2:
- M1(w) rejects
- M2(w) rejects
- So, M rejects w.

M(w):
1. Run M1(w). If accepts, accept.
2. Run M2(w). If accepts, accept, otherwise reject.

Where do we use decidability of M1, M2?

Closure for recognizable languages?

Theorem: The class of recognizable languages over fixed alphabet Σ is closed under union.

Can we adapt the proof from decidable languages?

Closure for recognizable languages?

Theorem: The class of recognizable languages over fixed alphabet Σ is closed under union.

Proof attempt: Let L_1 and L_2 be languages and suppose M_1 and M_2 are TMs recognizing these languages.

Construct the TM M as "On input w,

1. Run M_1 on input w. If M_1 accepts w, accept. Otherwise, go to 2.
2. Run M_2 on input w. If M_2 accepts w, accept. Otherwise, reject.“

This proof has a bug! Can you find it?

Closure for recogniz

Theorem: The class of recognizabl
alphabet Σ is closed under union.

Where does this proof fail?

a. M always halts
b. M never halts
c. M accepts some words outside L
d. M rejects some words in L
e. None of these

Proof attempt: Let L_1 and L_2 be lan
are TMs recognizing these languages.

Construct the TM M as "On input w,

1. Run M_1 on input w. If M_1 accepts w, accept. Otherwise, go to 2.
2. Run M_2 on input w. If M_2 accepts w, accept. Otherwise, reject.“

This proof has a bug!

Closure for recognizable languages?

Theorem: The class of recognizable languages over fixed alphabet Σ is closed under union.

Proof attempt: Let L_1 and L_2 be languages and suppose M_1 and M_2 are TMs recognizing these languages.

Construct the TM M as "On input w,

1. Run M_1 on input w. If M_1 accepts w, accept. Otherwise, go to 2.
2. Run M_2 on input w. If M_2 accepts w, accept. Otherwise, reject.“

ERROR: If $M_1(w)$ runs forever, we never get to run $M_2(w)$; we might not accept some words in L_2 !

Closure for recognizable languages?

Theorem: The class of recognizable languages over fixed alphabet Σ is closed under union.

We need a different proof strategy!

Challenge: how would you solve this?

Running TMs in parallel

- Main idea: **run both TMs in parallel**
- This is what your OS is doing when it is running several programs at the same time
- We will model it via running TMs for a limited number of steps

Step-bounded TM

- Let M be a TM, running on input w

- Define a new “bounded” TM - $M_{bdd}(w,t)$:
  - Run M(w) for at most t steps
  - If in these steps M accepted, accept; if it rejected, reject
  - If after t steps M(w) didn’t halt, then halt and reject

- Claim:

1. If $M_{bdd}(w,t)$ accepts then M(w) accepts
2. If M(w) accepts then for large enough t, $M_{bdd}(w,t)$ accepts

Closure for recognizable languages

Theorem: The class of recognizable languages over fixed alphabet Σ is closed under union.

Proof: Let L_1 and L_2 be languages and suppose M_1 and M_2 are TMs recognizing these languages.

Construct the TM M as "On input w,

1. t=1
2. While True:
   a) Run $M_{1,bdd}$(w,t). If it accepted, accept.
   b) Run $M_{2,bdd}$(w,t). If it accepted, accept.
   c) t=t+1

Closure for recognizable languages

M(w):
1. t=1
2. While True:
   (a) Run $M_{1,bdd}$(w,t). If it accepted, accept.
   (b) Run $M_{2,bdd}$(w,t). If it accepted, accept.
   (c) t=t+1

Proof of correctness:

(1) If M(w) accepts, then for some t,
either $M_{1,bdd}$(w,t) accepts or $M_{2,bdd}$(w,t) accepts.
This either M_1(w) accepts or M_2(w) accepts. So $w \in L_1 \cup L_2$

(2) If $w \in L_1$ then M_1(w) accepts after some finite number of steps t.
Then $M_{1,bdd}$(w,t) accepts and so M(w) accepts.

(3) If $w \in L_2$: analogous

Closure for recognizable languages

M(w):
1. t=1
2. While True:
   (a) Run $M_{1,bdd}$(w,t). If it accepted, accept.
   (b) Run $M_{2,bdd}$(w,t). If it accepted, accept.
   (c) t=t+1

Proof of correctness:

(1) If M(w) accepts, then for some t,
either $M_{1,bdd}$(w,t) accepts or $M_{2,bdd}$(w,t) accepts.
This ei… $\cup L_2$

(2) If w… …er of steps t.
Then M…

(3) If $w \in L_2$: analogous

M recognizes $L_1 \cup L_2$:
- If $w \in L_1 \cup L_2$ then M(w) accepts
- If $w \notin L_1 \cup L_2$ then M(w) loops

Closure under complement

- If L is decidable, is its complement decidable?

(a) Yes
(b) No
(c) It depends on L
(d) I don’t know

Closure under complement

- If L is recognizable, is its complement recognizable?

(a) Yes
(b) No
(c) It depends on L
(d) I don’t know

Closure

Good exercises – can't use without proof! (Sipser 3.15, 3.16)

The class of decidable languages is closed under

- Union
- Concatenation
- Intersection
- Kleene star
- **Complementation**

The class of recognizable languages is closed under

- Union
- Concatenation
- Intersection
- Kleene star

Church-Turing thesis

Sipser p. 183

- Wikipedia "self-contained step-by-step set of operations to be performed"
- "An algorithm is a finite sequence of precise instructions for performing a computation or for solving a problem."